

Lecture 1 — Programming for Performance

Jeff Zarnett, based on original by Patrick Lam

2026-10-01

Performance!

By this point, I'm certain you know what “programming” means, but we need to take a minute right off the top to define “performance”. This course is not about how to program when other people are watching (fun as that can be, as the popularity of Hackathons shows). What it's really about is making a program “fast”. Alright, but what does it mean for a program to be fast?

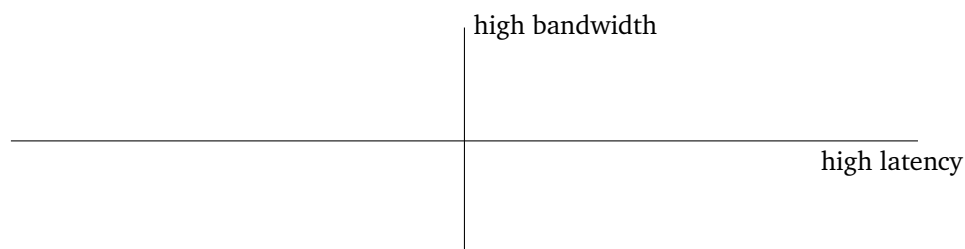
Let's think about the program execution as completion of some number of items – things to do. We have two concepts: items per unit time (bandwidth – more is better), and time per item (latency – less is better). Improving on either of these will make your program “faster” in some sense. In a way they are somewhat related: if we reduce the time per item from 5 s to 4 s it means an increase of 12 to 15 items per minute... if the conditions are right. Hopefully we could improve both metrics, but sometimes we'll have to pick one.

Items per unit time. This measures how much work can get done simultaneously; we refer to it as bandwidth. Parallelization – or doing many things at once – improves the number of items per unit time. We might measure items per time in terms of transactions per second or jobs per hour. You might still have to wait a long time to get the result of any particular job, even in a high-bandwidth situation; sending a truck full of hard drives across the continent is high-bandwidth but also high-latency.

Time per item. This measures how much time it takes to do any one particular task: we can call this the latency or response time. It doesn't tend to get measured as often as bandwidth, but it's especially important for tasks where people are involved. Google cares a lot about latency, which is why they provide the 8.8.8.8 DNS servers. (Aside relevant to a number of Capstone Design Projects I've seen: when dealing with users, 100ms is the maximum latency for systems that purport to respond instantaneously [Nie93].)

Examples. Say you need to make 100 paper airplanes. What's the fastest way of doing this?

Here's another example, containing various communications technologies:



We will focus on completing the items (doing useful work), not on transmitting information, but the above example illustrates the difference between bandwidth and latency.

Improving Latency

Although we'll mostly focus on parallelism in this course, a good way of writing faster code is by improving single-threaded performance. Unfortunately, there will be a limit to how much you can improve single-threaded performance; however, any improvements here may also help with the parallelized version. On the other hand, faster sequential algorithms may not parallelize as well. But let's take a look at some ways you can improve latency.

Profile the code. You can't successfully make your code faster if you don't know why it's slow. Intuition seems to often be wrong here, so run your program with realistic workloads under a profiling tool and figure out where all the time is going. This is a specific instance of one of my favourite rules of engineering: "Don't guess; measure".

Let's take a quick minute to visit <http://computers-are-fast.github.io/> and take a quiz on how fast computers can do certain operations [EM15]. Are the results surprising to you? Did you do really well or really badly? Chances are that you got some right and some wrong... and the ones that were wrong were not just a little wrong, but off by several orders of magnitude. Moral of the story is: don't just guess at what the slow parts of your code are. It's okay to have a theory as a starting point, but test your theory.

Do less work. A surefire way to be faster is to omit unnecessary work. Two (related) ways of omitting work are to avoid calculating intermediate results that you don't actually need and computing results to only the accuracy that you need in the final output.

Interesting to note: producing text output to a log file or to a console screen is surprisingly expensive for the computer. Sometimes one of the best ways to avoid unnecessary work is to spend less time logging and reporting. It might make debugging harder, yes, but once the code is correct (or close enough), removing the logging and debugging statements can actually make a difference. Especially in a multithreaded context, logging and debugging often incur synchronization cost.

A hybrid between "do less work" and "be smarter" is caching, where you store the results of expensive, side-effect-free operations (potentially I/O and computation) and reuse them as long as you know that they are still valid. Caching is really important in certain situations.

Be prepared. If you know something that the user is going to ask for in advance, you can have it at the ready to provide upon request. Example from that other job of mine [JZ]: users often want an Excel export of various statistics on their customs declarations. If the user asks for the report, generating it takes a while, and it means a long wait. If, however, the report data is pre-generated and stored in the database (and updated as necessary) then putting it in the Excel output file is simple and the report is available quickly.

Be smarter. You can also use a better algorithm. This is probably "low hanging fruit" and by the time it's time for P4P techniques this has already been done. But if your sorting algorithm is $\Theta(n^3)$ and you can replace it with one that is $\Theta(n^2)$, it's a tremendous improvement even if there are yet better algorithms out there. An improved algorithm includes better asymptotic performance as well as smarter data structures and smaller constant factors. Compiler optimizations (which we'll discuss in this course) help with getting smaller constant factors, as does being aware of the cache and data locality/density issues.

Sometimes you can find this type of improvement in your choice of libraries: you might use a more specialized library which does the task you need more quickly. The build structure can also help, i.e., which parts of the code are in libraries and which are in the main executable. It's a hard decision sometimes: libraries may be better and more reliable than the code you can write yourself. Or it might be better to write your own implementation that is optimized especially for your use case.

Improve the hardware. Once upon a time, it was okay to write code with terrible performance on the theory that next year's CPUs would make it acceptable, and spending a ton of time optimizing your code to run on today's processors was a waste of time. Well, those days seem to be over; CPUs are not getting much faster these days (evolutionary rather than revolutionary change). But sometimes the CPU is not the limiting factor: your code

might be I/O-bound, so you might be able to improve things dramatically by going to solid-state drives or non-volatile memory (e.g., Optane/3D XPoint); or you might be swapping out to disk, which kills performance (add RAM). Profiling is key here, to find out what the slow parts of execution are. When it comes down to it, spending a few thousand dollars on better hardware is often much cheaper than paying programmers to spend their time to optimize the code. (Programmers are super expensive.)

On using assembly. Not that long ago, compilers were not very smart and expert programmers could outsmart the compiler and produce better assembly by hand. This tends to be a bad idea these days. Compilers are going to be better at generating assembly than you are. Furthermore, CPUs may accept the commands in x86 assembly (or whatever your platform is) but internally they don't operate on those commands directly; they rearrange and reinterpret and do their own thing. Still, it's important to understand what the compiler is doing, and why it can't optimize certain things (we'll discuss that), but you don't need to do it yourself. However, giving hints to the compiler about, e.g., vector instructions can be helpful.

Anecdote time. A few years ago, I [JZ] was presented with a ticket that read as follows: "the report generation has been running for three hours; I think it's stuck." Turns out the report had not been running for that long; it reached a 30-minute time limit and the server had killed the task (and it just looked like it was running). So now I have a puzzle: how do I speed up this task to get it under the 30-minute time limit?

How does the report work? It selects the transactions for a given period from the database. Then for each transaction, it looks up the latest article data, recomputes the transaction's worth based on the most up to date currency exchange rate, and then stores the updated transaction in the database again.

Step one was to bring up the profiler and look at a few things. The slow steps were primarily database operations: retrieving exchange rates, retrieving the article data, and then storing all the transactions. Right, with this data, it's time to apply some strategies here.

Caching played a big role: the exchange rate data doesn't change for the report (it is run retroactively, with a date on the end of the last month, so the exchange rates are defined for that day rather than floating). So retrieving the exchange rate 500 times can be cut down to once per currency. Caching was also important for the articles; an article might be used dozens of times, so loading it from the database repeatedly is also a waste of time. Also, I could select all the articles at once rather than each one as encountered.

How about doing less work? For one thing, instead of pulling all the fields of the article from the database, why not just get the five that are actually needed? And in saving the transactions, what if we only update the parts that changed rather than update the full transaction and all its parts?

Ultimately, these techniques combined brought the report time down under 30 minutes and it can now run to completion.

Doing more things at a time

Rather than, or in addition to, doing each thing faster, we can do more things at a time.

Why parallelism?

While it helps to do each thing faster, there are limits to how fast you can do each thing. The (rather flat) trend in recent CPU clock speeds illustrates this point. Often, it is easier to just throw more resources at the problem: use a bunch of CPUs at the same time. We will study how to effectively throw more resources at problems. In general, parallelism improves bandwidth, but not latency. Unfortunately, parallelism does complicate your life, as we'll see.

Different kinds of parallelism. Different problems are amenable to different sorts of parallelization. For instance, in a web server, we can easily parallelize simultaneous requests. On the other hand, it's hard to parallelize

a linked list traversal. (Why?)

Pipelining. A key concept is pipelining. All modern CPUs do this, but you can do it in your code too. Think of an assembly line: you can split a task into a set of subtasks and execute these subtasks in parallel.

Hardware. To get parallelism, we need to have multiple instruction streams executing simultaneously. We can do this by increasing the number of CPUs: we can use multicore processors, SMP (symmetric multiprocessor) systems, or a cluster of machines. We get different communication latencies with each of these choices.

We can also use more hardware, like vector processing units built into all modern chips (SIMD) or graphics processing units (GPUs).

Difficulties with using parallelism

You may have noticed that it is easier to do a project when it's just you rather than being you and a team. The same applies to code. Here are some of the issues with parallel code.

First, some domains are “embarrassingly parallel” and these problems don't apply to them; for these domains, it's easy to communicate the problem to all of the processors and to get the answer back, and the processors don't need to talk to each other to compute. The canonical example is Monte Carlo integration, where each processor computes the contribution of a subrange of the integral.

I'll divide the remaining discussion into limitations and complications.

Limitations. Parallelization is no panacea, even without the complications that I describe below. Dependencies are the big problem.

First of all, a task can't start processing until it knows what it is supposed to process. Coordination overhead is an issue, and if the problem doesn't have a succinct description, parallelization can be difficult. Also, the task needs to combine its result with the other tasks.

“Inherently sequential” problems are an issue. In a sequential program, it's OK if one loop iteration depends on the result of the previous iteration. However, such formulations prohibit parallelizing the loop. Sometimes we can find a parallelizable formulation of the loop, but sometimes we haven't found one yet.

Finally, code often contains a sequential part and a parallelizable part. If the sequential part takes too long to execute, then executing the parallelizable part on even an infinite number of processors isn't going to speed up the task as a whole. This is known as Amdahl's Law, and we'll talk about this in a few weeks.

Complications. It's already quite difficult to make sure that sequential programs work right. Making sure that a parallel program works right is even more difficult.

The key complication is that there is no longer a total ordering between program events. Instead, you have a partial ordering: some events A are guaranteed to happen before other events B , but many events X and Y can occur in either the order XY or YX . This makes your code harder to understand, and complicates testing, because the ordering that you witness might not be the one causing the problem.

Two specific problems are data races and deadlocks.

- A *data race* occurs when two threads or processes both attempt to simultaneously access the same data, and at least one of the accesses is a write. This can lead to nonsensical intermediate states becoming visible to one of the participants. Avoiding data races requires coordination between the participants to ensure that intermediate states never become visible (typically using locks).

- A *deadlock* occurs when none of the threads or processes can make progress on the task because of a cycle in the resource requests. To avoid a deadlock, the programmer needs to enforce an ordering in the locks. Or use some other strategies as we have discussed in previous courses.

Another complication is stale data. Caches for multicore processors are particularly difficult to implement because they need to account for writes by other cores.

Scalability

It gets worse. Performance is great, but it's not the only thing we're interested in. We also care about *scalability*: the trend of performance with increasing load. A program generally has a designed load (e.g., we are expecting to handle x transactions per hour). A properly designed program will be able to meet this intended load. If the performance deteriorates rapidly with increasing load (that is, the number of operations to do), we say it is *not scalable* [Liu09]. This is undesirable, of course, and for the most part if we have a good program design it can be fixed. If we have a bad program design, then no amount of programming for performance techniques is going to solve that ("rearranging deck chairs on the Titanic").

The things we're going to look at in this course are ways to meet x or even raise the value of x . Even the most scalable systems have their limits, of course, and while higher is better, nothing is infinite. There's only so much we can do to push it, but chances are we can make some serious progress if we make the effort.

Laws of Performant Software

Suppose you want to write fast programs and you like checklists and handy rules. If so, you are in luck, because there are Crista's Five Laws of Performant Software [Lop16].

1. Programming language $\ll$ Programmers' awareness of performance. There is no programming language that is magic, whether good or evil. All the major programming languages allow you to write programs that perform well or badly.

There's a lot of C-elitism in the world, and then there's the back-in-my-day-sonny people who claim assembly was best, and they also had to walk to school in the snow, uphill both ways. High level languages give you lots of options... Do I use an array? A vector? A list? And yes, some of the fancy tools you get are syntactic sugar: they are convenient from the programmer's point of view, but what do they do behind the scenes? If the performance is not what you expect, there is probably a better way to do it in the high level language.

I'll add my own asterisk on this rule: some languages lend themselves better to parallelization than others. A language may force a certain way of thinking, based on its rules (e.g., functional programming languages). But there is no reason why the way of thinking can't be applied in another language.

2. $d(f^\tau(x), f^\tau(y)) > e^{\alpha\tau} d(x, y)$ or small details matter. This complicated formula is from the butterfly effect (chaos theory). If two versions of the code are x and y , the difference between the performance outcomes $f(x)$, $f(y)$ is much larger than the difference between the code.

A small code change can have a huge impact. Did you fix a memory leak? The addition of one `free()` call is a single line code change but can, in the long run, have a dramatic impact on performance. Is caching used properly? Can you use a faster serialization algorithm?

Basically: don't overlook the small stuff. It's tempting to think that huge major architectural changes are the solution to everything; but there are plenty of gains to be found in the small things.

3. corr(performance degradation, unbounded resource usage) > 0.9. There is a very high correlation between performance degradation and unbounded use of resources. Oftentimes we focus on functionality: the software must have the following 847 251 features! But if you want a program that scales you need to think in terms of operation, not functionality.

Resources need to be limited. If there aren't hard limits, eventually a resource will be exhausted. If the program starts threads, use a thread pool and the thread pool should have a fixed size. Is there a cache? It needs a maximum size. If you need to read input, don't use a function that reads an entire line (of arbitrary length). Furthermore your program needs design effort given to what happens when resources are exceeded. So you decide to set a request queue size; once that queue is full, further requests are rejected in some well-understood manner.

4. Performance improvements = log(controlled experiments) If you want your code to be faster you have to know why it is slow. It's okay not to know the answers, but not knowing how to find out is a problem. Don't guess; measure.

5. N*bad != good. No amount of nodes, cores, memory, etc., will save you from poorly-written code. Throwing more hardware at the problem is expensive and ineffective in the long term. Bad code is still bad no matter how much hardware it runs on.

Rust

Previous courses you have taken have likely used C and C++ as systems languages. ECE 459 used to as well! It's possible that one of those was your first programming language and perhaps even the one you've used the most. The languages themselves have their strengths and weaknesses, of course, but there's no denying that these languages come without some of the niceties found in other languages like clever static type checking and garbage collection.

The nature of the languages makes it hard, or even impossible, to write code that is fast, correct, and secure. The focus of this course hasn't been on security. But in many cases, writing insecure fast code isn't the right thing. Is it even possible to write secure C and C++?

Maybe not. The usual arguments are something along the lines of experience. Experience isn't it either, given this quotation from Robert O'Callahan: "I cannot consistently write safe C/C++ code."¹ (17 July 2017) (Holds a PhD in CS from Carnegie Mellon University; was Distinguished Engineer at Mozilla for 10 years; now at Google; etc.)

What about use of better tools and best practices? March 2019: disclosure of Chrome use-after-free vulnerability²; 0-day attacks observed in the wild. Google implements best practices, and has all the tools and developers that money can buy!

Much of the advice about how to avoid these problems comes down to "try harder", which is...not helpful. If the strategy is just dragging people and saying that they need to pay more attention, or be more careful, or other similar phrase...this is going to constantly be an uphill battle. Expecting people to be perfect and make no mistakes is unrealistic. What we want here is to make mistakes difficult-to-impossible. (Mitigating the effects of mistakes is another good strategy.)

A lot of the problems we frequently encounter are the kind that can be found by Valgrind, such as memory errors or race conditions. Other tools like code reviews and Coverity (static analysis defect-finding tool) exist. These are good, but not perfect. Valgrind, for example, only reports errors that it actually sees executed, so until and unless every function and every code path is run, it might not report a problem. Static analysis tools try to track down problems at compile-time, and that seems like a lot better of a solution.

I like to solve not just an individual problem, but an entire class of problems all at once. A somewhat-recent

¹<https://robert.ocallahan.org/2017/07/confession-of-cc-programmer.html>

²<https://security.googleblog.com/2019/03/disclosing-vulnerabilities-to-protect.html>

example: if you change the contents of a list in a background thread while it's being rendered, the rendering thread will fail because the list has changed. I can fix the line of code so the list manipulation does not happen during rendering, and that fixes it once, but not forever: in the future, another person (or even Future Me, having forgotten my previous experience) could write code that calls this function from a background thread. There's no good way (in Java, sadly) to make it so invoking this function incorrectly is a compile-time error, so the best I can do is set a trap in it that throws an error if called inappropriately, so that the responsible developer will find what they did wrong during development and testing. Compile-time error checking is preferable to run-time, because the cost of fixing it is lower if it is caught earlier in the process.

A broader perspective: as you know from your co-op work terms, industrial codebases range from hundreds of thousands to millions (and more) of lines of code. You can fix localized problems, like an object that is allocated in a method, doesn't escape, and isn't freed. Localized problems don't require you to look across large parts of the codebase. But the rendering problem above requires non-local knowledge: you add some code that does list manipulation. While you're doing that, you don't see the requirement to not be called during rendering. You can't keep all of the conventions in your head. Tool support is necessary.

This brings us to Rust. It is an alternative to C/C++, incorporating many good ideas from C++ (e.g., RAII, references) and sometimes taking them up a notch. It is a new-school secure systems programming language used by Mozilla's Project Quantum. A design goal of this language is to avoid issues with memory allocation and concurrency. It does so by checking things at compile time that most languages check only at runtime, if at all.

Tips and caveats. Like any designed artifact, Rust makes tradeoffs. Sometimes the tradeoffs will work out in your favour, and sometimes they won't. In particular, you will find some things harder to code in Rust than in C/C++. However, they are also more likely to be correct. Is that worth it? Depends on the context: are you writing throwaway code? A prototype? Code that is destined for production in a critical system?

Even though you've been writing code for at least a couple of years, if you are new to Rust, you will have frustrating moments. It may seem like Rust is out to frustrate you, but Rust's developers have put a lot of effort into producing error messages that try to help. And, as always, please use Piazza and otherwise ask colleagues and course staff for help.

There's the saying "If you know one programming language, you know them all". This saying is true to a first approximation, at least for first-year programming. C/C++/C#/Java are reasonably similar and it's possible to write the same sort of code in at least the object-oriented languages. But the saying is not fully true, for a bunch of reasons. Writing *idiomatic* code in a language is different from writing working code, and I'll encourage you to learn Rust as it is and to write Rust code rather than C++ code masquerading as Rust code. In particular, Rust also admits influences from functional languages like Haskell and OCaml.³

Rust isn't always faster than C++. We'll talk about a specific example, in the context of exceptions (Rust doesn't have them), in Lecture 3.

The Roadmap

First thing we will do is talk about Rust in some more detail. We learned a little bit about why Rust, but we also acknowledge that it's very likely that you have limited to no experience with the language at all. The intention is not to teach you fundamentals of programming, but instead to guide you on the Rust philosophy so you can apply it in the assignments.

You have a program and you want to make it fast. To understand what's going on we will need some baseline understanding of hardware: architecture, caches, branch prediction. Understanding those will tell you what the pitfalls are that can make your program slow.

³Yes, C++, C# and Java all have incorporated functional features now too, but they're not central to how people write in these languages typically.

An easy way to get a performance boost is parallelizing your program: use threads for a big performance boost, mitigate the risks (with locking) but also do it well.

Then when that's mined out you can start thinking about speculation and also about trying to speed up your single thread performance. You can think about going to OpenCL if you have the right kind of task for it, but conversion is hard and the overhead is large.

If you've done all the things that are sure to make improvement it's time to really dig in with the profiling tools to find where and what to focus on next. And when all (reasonable) improvements have been made, then it's time to make your code work using multiple machines (and apply some queueing theory to find out how many servers you need!).

Acknowledgements

Thanks to Sunjay Varma for general comments as well as specific Rust corrections.

References

- [EM15] Julia Evans and Kamal Marhubi. Do you know how much your computer can do in a second?, 2015. Online; accessed 28-October-2015. URL: <http://computers-are-fast.github.io/>.
- [Liu09] Henry H. Liu. *Software Performance and Scalability: A Quantitative Approach*. John Wiley & Sons, 2009.
- [Lop16] Crista Videira Lopes. Laws of performant software, 2016. Online; accessed 28-December-2016. URL: <http://tagide.com/blog/advice/laws-of-peformant-software/>.
- [Nie93] Jakob Nielsen. Response times: The 3 important limits, January 1993. Online; accessed 5-December-2017. URL: <https://www.nngroup.com/articles/response-times-3-important-limits/>.